

Full Architecture Overview and Implementation Strategy

1. Architecture Overview

This section outlines a cohesive, modern software architecture that automates the linking of Jira tickets to relevant code segments within large repositories, leveraging advanced AI and AWS cloud services.

A. Key Architectural Components

1. VS Code Extension (Frontend)

- User interface facilitating Jira ticket review, code search, and association
- Integrated with MCP (Model Context Protocol) to communicate with backend AI services [code.visualstudio](#)
- Allows inline recommendations, code segment highlighting, and status updates

2. MCP Server (Backend)

- Acts as a bridge between VS Code extension and AWS-based AI services
- Parses code, manages interactions with Jira REST API, and orchestrates AI tasks

3. AI-powered Code Analysis

- Uses Amazon Q for code embedding generation, enabling semantic similarity search [aws.amazon+1](#)
- Employs Amazon Bedrock for context-aware recommendations (e.g., suggested bug fixes, refactoring)

4. Vector Storage and Similarity Search

- Persists and queries code embeddings using Amazon S3 Vectors (purpose-built for scalability) [aws.amazon+1](#)
- Supports fast, high-volume similarity search across codebase

5. Issue Management Integration

- Connects with Jira via REST API, enabling automatic updates, comments, and traceability links [github+2](#)
- Enables bi-directional traceability between tickets and code (supports audits, compliance, and change management) [parasoft+2](#)

6. Optional Knowledge Graph Integration

- Augments semantic search by representing relationships between code segments, tickets, requirements, API docs, etc. [openreview+3](#)

B. Data Flow and Sequence

1. **Ticket Initiation:** Developer opens a Jira ticket in VS Code extension.

2. **Semantic Code Search:** MCP server extracts ticket context and queries embeddings in S3 vectors to find related code segments.
3. **Recommendations and Linking:** Amazon Bedrock analyzes the context, providing recommended code changes and justifications; developer reviews and applies links.
4. **Traceability Update:** Extension updates Jira ticket (comments, related code links, etc.); optionally updates knowledge graph and traceability matrix.[valgenesis+2](#)
5. **Continuous Feedback:** As code/tickets/requirements evolve, embeddings and links are refreshed to maintain accurate traceability.

C. Technology Choices

- **TypeScript:** VS Code extension frontend
- **Node.js:** MCP server
- **Amazon Q API / Amazon Bedrock:** Embedding generation, LLM-powered recommendations
- **Amazon S3 Vectors:** Vector storage & similarity lookups
- **Jira REST API:** Ticket operations
- **AWS Lambda (Optional):** For workflow automation, scaling sub-tasks
- **Knowledge Graph (Optional):** Neo4j, RDF stores, or custom AWS Neptune/KG

2. Step-by-Step Implementation

1. **Set Up Dev Environment:** Scaffold VS Code extension, configure TypeScript and AWS SDK.[dev+1](#)
 2. **Configure MCP Server:** Implement code to communicate bidirectionally with extension and connect to AWS APIs.
 3. **Jira API Integration:** Authenticate and implement functions to read, update, and comment on tickets.[atlassian+2](#)
 4. **Codebase Parsing:** Develop or integrate parsers for code segment extraction (functions, classes, modules).[adasci](#)
 5. **Code Embedding Generation:** Use Amazon Q to convert code segments to vector representations.[aws.amazon+1](#)
 6. **Vector Storage & Similarity Search:** Store embeddings in S3 vector buckets and implement retrieval for similarity search.[aws.amazon+2](#)
 7. **AI Recommendation Engine:** Leverage Bedrock to provide ticket-aware code suggestions and context.[aws.amazon+1](#)
 8. **Bi-directional Traceability:** Automate requirements-to-code links and propagate them to Jira/comments/RTM.[model-engineering+3](#)
 9. **Optional Knowledge Graph Construction:** Model and maintain semantic relationships for advanced queries and traceability.[arxiv+3](#)
-

Research Work: Current State and Enhancement Opportunities

1. State-of-the-Art Research

a. Semantic Code Search and Traceability

- Deep code embeddings (BERT, CodeBERT, GPT-based) outperform traditional keyword-based search for linking tickets to code.[arxiv+3](#)
- LLMs such as Claude 3.5 Sonnet and GPT-4o, when evaluated for documentation-to-code traceability, show F1-scores above 79%—far exceeding IR methods but requiring human review due to some context-mismatch errors.[arxiv+1](#)
- Automated traceability matrices are helping teams manage requirement-code-test links, reduce compliance risk, and enable effective impact analysis.[abstracta+4](#)

b. AI in Issue Management

- BERT and other transformer models are being researched for predicting issue links and dependencies in Jira and other trackers.[acm+1](#)
- ChatGPT and similar LLMs are being studied for their role in issue resolution, brainstorming, and code generation, though developers are wary of hallucinated code and use AI mainly for ideation.[arxiv](#)

c. Knowledge Graphs and Augmented Reasoning

- Knowledge graphs are increasingly used for representing relationships across artifacts, improving code reuse/retrieval, and augmenting semantic search.[sciencedirect+3](#)
- Combined RAG (Retrieval Augmented Generation) systems leverage embeddings and knowledge graphs for better requirement-to-code traceability and context-aware code suggestions.[model-engineering+1](#)

d. Developer Productivity Tools

- The landscape in 2025 shows AI tools (assistants, coding agents, semantic search) are widely adopted (70-90% across teams), but concerns persist about explainability, code quality, and context nuance.[baytechconsulting+3](#)
- Collaboration between human engineers and AI is emphasized to overcome ambiguity/specification gaps and ensure trustworthy results.[ibm+3](#)

2. How to Write About This Project (Research Section Structure)

a. Literature Review

- Discuss how previous works tackled code search and ticket-code linking, summarizing their approaches and limitations.[arxiv+4](#)
- Highlight the evolution from keyword-based and IR approaches toward semantic embeddings, knowledge graphs, and LLM-powered linking.[acm+4](#)

b. Problem Statement

- Identify the gaps: manual linking is error-prone, time-consuming, and fails in large, dynamic codebases. Pure semantic/vector search improves, but is not perfect—context and intent still present challenges.[devops+4](#)

c. Methodological Innovations

- Explain your project's modular architecture, which leverages SOTA methods: LLM-powered embeddings, transformer-based link prediction, S3 Vectors for scalable storage, knowledge graphs for advanced reasoning, and bi-directional traceability for full lifecycle linking.[perforce+5](#)
- Justify choice of AWS stack for scalability and integration ease.

d. Experimental Design

- Plan quantitative metrics: link prediction accuracy, code search speed, developer satisfaction, compliance coverage (RTM completeness).
- Include qualitative user feedback and human-in-the-loop error review to assess hallucination/context gaps.

e. Research Gaps and Enhancement Opportunities

- Error analysis: Where do LLMs fail? Name-matching, overgeneralization, missing context.[reddit+2](#)
- Human-AI collaboration: How to integrate explainability and feedback loops for trustworthy traceability.[news.mit+1](#)
- Multi-modal semantic search: Combining embeddings, KG reasoning, and manual review to deliver more robust results.
- Automation expansion: Integrate traceability beyond tickets-code (requirements, tests, release notes, pipeline logs).[scitepress+2](#)
- Scalability: Handling large enterprise codebases without performance degradation.[aws.amazon+1](#)

f. Future Directions

- Explore integrating open-source models and custom transformers for specialized code domains.
- Develop tools for dynamic, auto-updating knowledge graphs—moving from static links to living traceability ecosystems.
- Collaborate with end users to refine AI explainability, feedback workflows, and trust mechanisms.

Sample Research Paper Structure (Domain-Specific Recommendations)

1. Introduction

- Motivation, state of current tools, significance of automation and AI in this context.

2. Related Work

- Comparative review of peer-reviewed papers and recent advances: BERT for issue linking, transformer-based code search, knowledge graph approaches, automated RTM systems.[sciencedirect+8](#)

3. Methodology

- Architectural diagram, data flow, technology stack rationale.
- Algorithms/models used for embedding, similarity search, link prediction.
- Knowledge graph and traceability design.

4. Implementation

- AWS infrastructure integration, modular VS Code extension, MCP architecture.
- Practical techniques for scaling, error recovery, and explainability.

5. Evaluation

- Quantitative: search accuracy, link precision/recall, compliance coverage.
- Qualitative: user interviews, developer adoption/satisfaction.

6. Discussion

- Key research findings, limitations (AI hallucinations, context gaps), open challenges.
- Recommendations for best practices, feedback loop design, and enhancement paths.

7. Conclusion and Future Work

- Summary of contributions.
- Roadmap for improved multi-modal traceability, dynamic knowledge graph integration, and robust developer-AI collaboration.

Summary Table: Architecture & Research Enhancements

Component	Latest Research	Enhancement Recommendations
Code Embedding	BERT, CodeT5, GPT sciencedirect+1	Multi-modal embeddings, custom domain models
Similarity Search	S3 Vectors, Qdrant adasci+1	Hybrid methods: semantic + graph, scalable batch querying
Issue Linking	LLM prediction arxiv+1	Human-in-the-loop review, context-aware graphs
Knowledge Graphs	KG/RAG frameworks openreview+1	Live updating, auto-integration of new relations

Component	Latest Research	Enhancement Recommendations
Traceability Matrix	Automated RTM parasoft+4	End-to-end pipeline linkage, dynamic validation coverage
Developer Productivity	DORA/SPACE frameworks getdx+1	Measure AI adoption, feedback-driven workflow improvement
Error Analysis	LLM hallucinations arxiv+2	Explainability, active error collection, model refinement

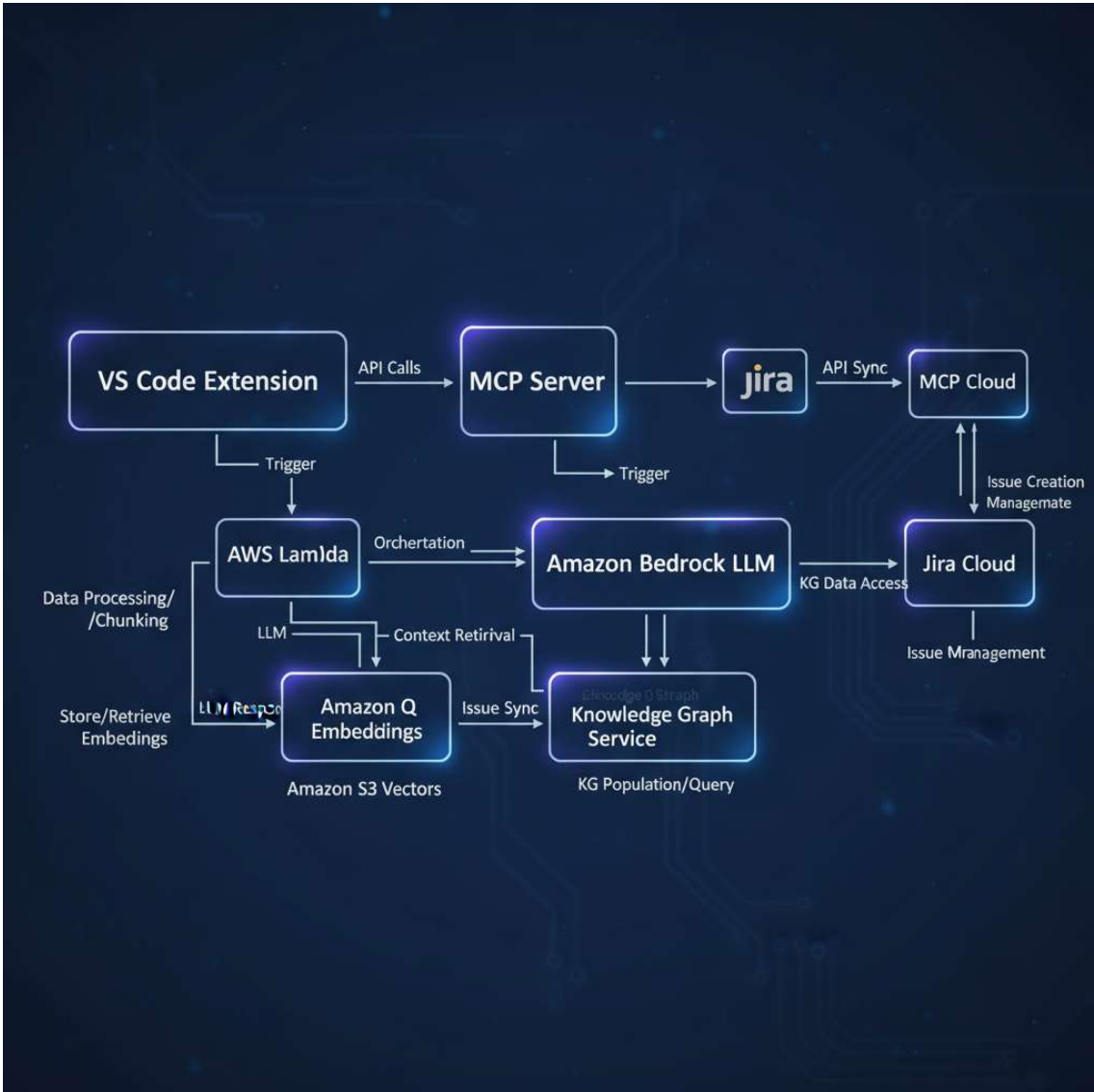
This approach will prepare your project for publication-grade documentation and equip it for enterprise-scale deployment, ensuring it stands at the forefront of software engineering research in AI-assisted issue-code traceability.

1. <https://code.visualstudio.com/api/extension-guides/ai/mcp>
2. <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/software-dev.html>
3. <https://aws.amazon.com/q/developer/>
4. <https://aws.amazon.com/blogs/aws/introducing-amazon-s3-vectors-first-cloud-storage-with-native-vector-support-at-scale/>
5. <https://aws.amazon.com/s3/features/vectors/>
6. <https://github.com/KnisterPeter/vscode-jira>
7. <https://support.atlassian.com/bitbucket-cloud/docs/jira-issues-in-vs-code/>
8. <https://support.atlassian.com/bitbucket-cloud/docs/use-the-atlassian-for-vs-code-extension/>
9. <https://www.parasoft.com/blog/requirements-management-and-the-traceability-matrix/>
10. <https://abstracta.us/blog/testing-strategy/requirements-traceability-matrix-your-qa-strategy/>
11. <https://testrigor.com/blog/requirement-traceability-matrix-rtm/>
12. <https://openreview.net/forum?id=8crfUZEGWP>
13. <https://arxiv.org/html/2505.14394v1>
14. <https://www.ibm.com/think/topics/knowledge-graph>
15. <https://cacm.acm.org/research/knowledge-graphs/>
16. <https://www.valgenesis.com/blog/the-benefits-of-automating-your-requirements-traceability-matrix-rtm>
17. <https://arxiv.org/html/2509.11238v1>
18. <https://dev.to/fabrikapp/how-to-make-a-vs-code-extension-using-typescript-a-step-by-step-guide-1hp6>

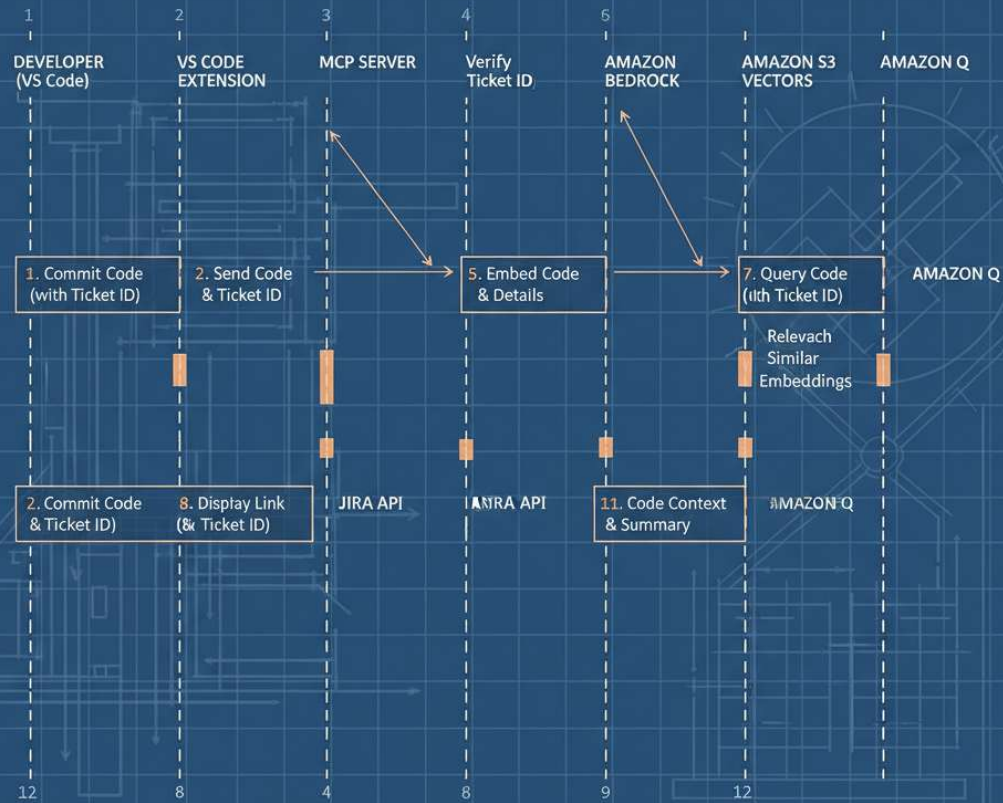
19. <https://docs.aws.amazon.com/toolkit-for-vscode/latest/userguide/setup-toolkit.html>
20. <https://adasci.org/code-search-with-vector-embeddings-using-qdrant-vector-database/>
21. <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html>
22. <https://docs.aws.amazon.com/bedrock/latest/userguide/api-inference-examples-claude-messages-code-examples.html>
23. <https://docs.aws.amazon.com/bedrock/latest/userguide/getting-started-api-ex-python.html>
24. <https://model-engineering.info/publications/papers/ER24-Requirements2Code.pdf>
25. <https://dl.acm.org/doi/10.1145/3696630.3730569>
26. <https://www.perforce.com/resources/alm/requirements-traceability-matrix>
27. <https://www.sciencedirect.com/science/article/abs/pii/S0950584923001829>
28. <https://arxiv.org/abs/2206.07182>
29. <https://arxiv.org/abs/2201.11313>
30. <https://www.sciencedirect.com/science/article/abs/pii/S0957417425003847>
31. <https://www.sciencedirect.com/science/article/pii/S1319157824002076>
32. <https://arxiv.org/html/2506.16440v1>
33. https://dl.acm.org/doi/abs/10.1007/978-3-031-70242-6_13
34. <https://arxiv.org/abs/2402.03735>
35. <https://www.baytechconsulting.com/blog/ai-and-software-development-2025>
36. https://www.reddit.com/r/programming/comments/1mgry93/the_state_of_software_development_in_2025/
37. <https://www.devopsdigest.com/gartner-top-strategic-trends-in-software-engineering-for-2025-and-beyond>
38. <https://sam-solutions.com/blog/software-development-trends/>
39. <https://www.ibm.com/think/news/ai-write-code-can-beat-software-engineers>
40. <https://devops.com/ai-coding-new-research-shows-even-the-best-models-struggle-with-real-world-software-engineering/>
41. <https://news.mit.edu/2025/can-ai-really-code-study-maps-roadblocks-to-autonomous-software-engineering-0716>
42. <https://arxiv.org/html/2503.22625v1>
43. https://www.reddit.com/r/ArtificialIntelligence/comments/1jwfm94/recent_study_reveals_performance_limitations_in/
44. <https://allthingsopen.org/articles/ai-code-assistants-limitations>
45. <https://www.scitepress.org/Papers/2024/126267/126267.pdf>

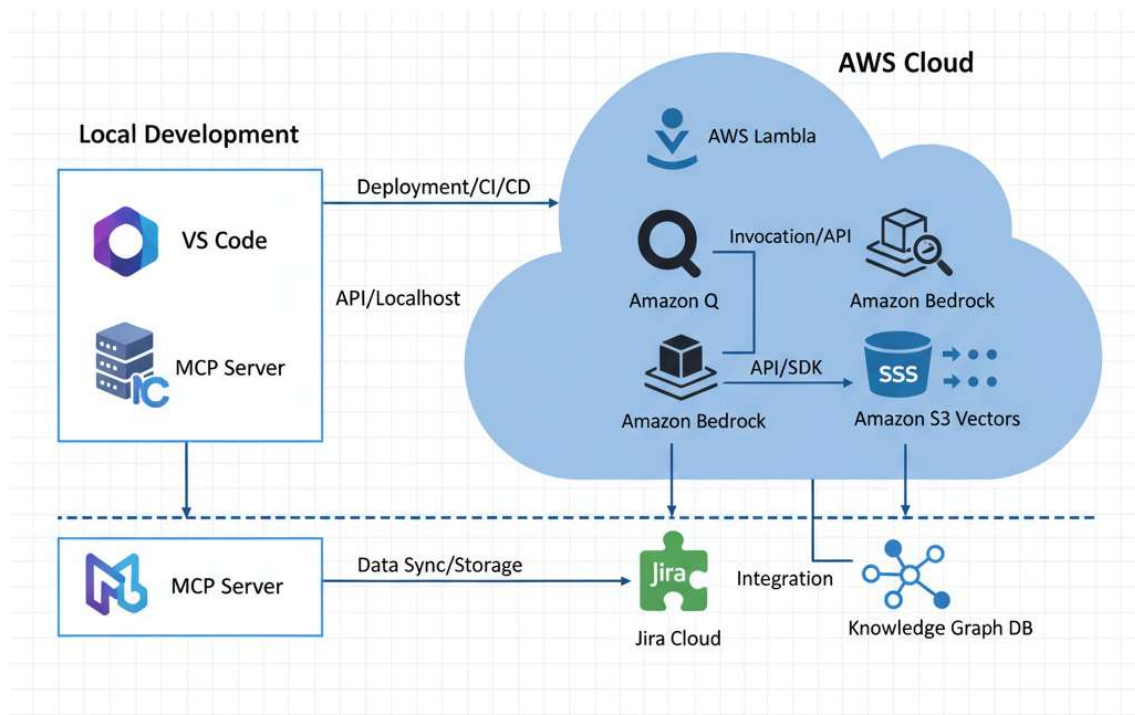
46. <https://getdx.com/research/overview/>
47. <https://research.google/pubs/what-predicts-software-developers-productivity/>
48. <https://brainhub.eu/library/software-developer-age-of-ai>
49. <https://www.lumar.io/blog/best-practice/semantic-search-explained-vector-models-impact-on-seo/>
50. <https://www.sciencedirect.com/science/article/pii/S0957417423004864>
51. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11795448/>
52. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
53. <https://www.sciencedirect.com/science/article/pii/S0167642321000216>
54. https://www.reddit.com/r/learnmachinelearning/comments/1acxy85/best_practices_for_semantic_search_on_200k/
55. <https://newsletter.pragmaticengineer.com/p/software-engineering-with-llms-in-2025>
56. <https://dev.to/imsushant12/embeddings-vector-databases-and-semantic-search-a-comprehensive-guide-2j01>
57. <https://conf.researchr.org/details/profes-2024/profes-2024-short-papers-and-posters/2/Supporting-value-aware-software-engineering-through-traceability-and-value-tactics>
58. <https://www.turing.ac.uk/research/interest-groups/knowledge-graphs>
59. <https://www.youtube.com/watch?v=eTBCHSTvjPg>
60. <https://www.computer.org/csdl/journal/ts/2025/03/10855629/23QQVxBVRqU>
61. <https://www.ketryx.com/capabilities/traceability>
62. <https://www.dagstuhl.de/24161>
63. <https://www.port.io/blog/developer-productivity-tools>
64. <https://lemon.io/blog/software-engineering-job-market/>
65. <https://www.atlassian.com/blog/loom/developer-productivity-tools>
66. <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/yes-you-can-measure-software-developer-productivity>
67. <https://softjourn.com/insights/ai-coding-tools>
68. <https://ser-rse-bridge.github.io>
69. https://www.reddit.com/r/ExperiencedDevs/comments/194gjqk/developer_productivity_to_ols/

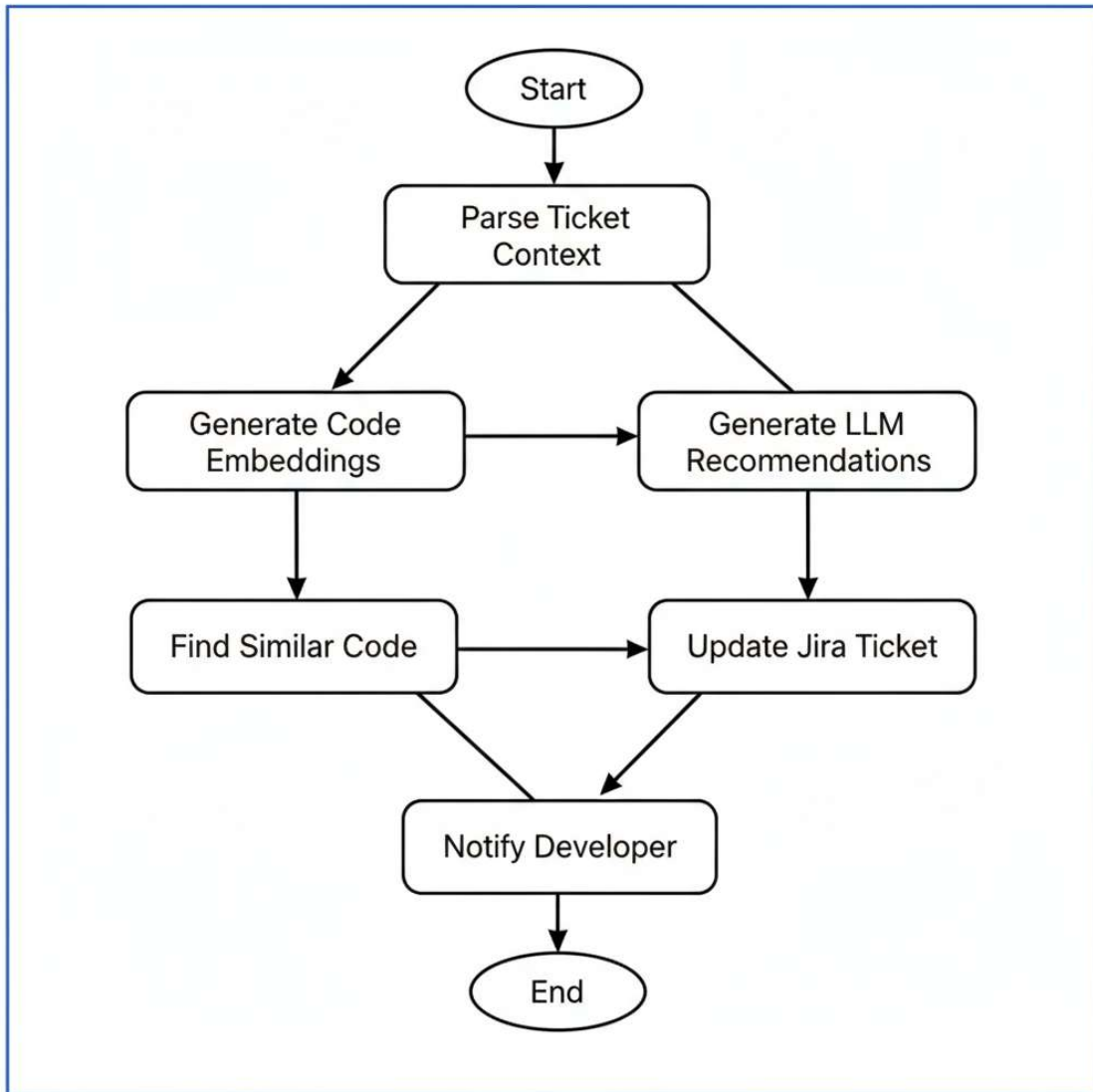
System Design Diagram



TICKET-CODE LINKING FLOW







AI Code-Ticket Linking System ERD

PK: Primary Key

